ADVANCED BIG DATA · CAPSTONE PROJECT
National School of Applied Sciences — Al Hoceima · 2025–2026

TaaSim

Transport as a Service — Urban Mobility Platform

Casablanca, Morocco · 2025–2026

Course	Advanced Big Data — Final Capstone Project
School	National School of Applied Sciences — Al Hoceima (ENSAH)
Academic Year	2025–2026
Proposed by	Mohamed El Marouani
Duration	8 weeks · weekly lab sessions (~3 hours each)
Team size	2–3 students per group
Tech stack	Kafka · Spark · Flink · MinIO · Cassandra · Grafana
Datasets	Porto Taxi Trajectories (ECML 2015) + NYC TLC (Kaggle)
Framing	Startup pitch at final demo — 'You are co-founders'

"Build the data platform that moves Casablanca."

You are not just students. You are co-founders.

1. Context & Problem Statement

Casablanca is the economic capital of Morocco and home to over 4 million inhabitants. Despite significant investment in formal transit infrastructure — the BRT corridor along Boulevard Mohammed VI, the ONCF suburban rail, and thousands of licensed taxis — urban mobility remains deeply fragmented. The core problem is not a shortage of vehicles: it is the total absence of a data layer connecting supply to demand.

Pain Point	Reality in Casablanca
No shared data layer	Grand taxis, petits taxis, and informal minibuses operate with no GPS tracking, no digital booking, no shared schedule
Demand blindness	Drivers cruise looking for passengers; passengers wait with no visibility of vehicle availability
No interoperability	Formal transit (ONCF, BRT) and informal taxis share no data, no ticketing, no trip planner
Cash-only payments	Zero payment data means zero trip history, zero analytics, zero personalization
Underserved periphery	New districts (Bouskoura, Ain Sebaâ, Sidi Moumen) grow faster than routes are planned

TaaSim is a Transport-as-a-Service platform that treats urban mobility as a data engineering problem. By ingesting GPS vehicle streams, processing citizen trip reservations in real time, and applying batch analytics and machine learning to historical patterns, TaaSim can match riders to vehicles dynamically, forecast demand surges, and give city planners a unified analytical view of the mobility network.

> **Academic Framing**
> Each team is a startup co-founding a mobility platform for Casablanca.
> The pipeline you build is your product — your competitive moat is the data you accumulate and process.
> Week 8 demo = seed-round investor pitch. Technical excellence + product thinking, both required.

2. Datasets

A critical real-world constraint: no public open dataset exists specifically for Casablanca taxi or transport trips. Researchers studying Casablanca mobility have had to use Google Traffic API scraping or smartphone GPS traces as a proxy — there is no official open data release. This is itself an important lesson for students about data engineering in emerging markets.

TaaSim uses two well-established open academic datasets as proxies, and a lightweight simulation layer to inject real-time stream behavior. Both datasets are freely available and widely used in research.

2.1 Porto Taxi Trajectories — Primary Dataset

Attribute	Detail
Full name	Taxi Service Trajectory — ECML/PKDD 2015 Challenge Dataset
Source	UCI Machine Learning Repository + Kaggle (CC BY 4.0 — free for academic use)
URL	https://www.kaggle.com/c/pkdd-15-predict-taxi-service-trajectory-i
Volume	~1.7 million completed taxi trips · ~1.5 GB compressed CSV
Time period	July 2013 – June 2014 (12 months)
Fleet size	442 taxis operating in Porto, Portugal
Key fields	TRIP_ID · CALL_TYPE · TAXI_ID · TIMESTAMP · POLYLINE (GPS every 15 sec) · DAY_TYPE
CALL_TYPE	A = dispatched from central; B = picked up from taxi stand; C = street hail
Why this dataset?	Call types A/B/C mirror exactly how Casablanca grand taxis and petits taxis are hailed. Porto is a comparable mid-size city. Dataset is the most-cited taxi trajectory dataset in academic ML research.

Casablanca Mapping Strategy

Porto's 22 city zones are remapped to Casablanca's 16 arrondissements (provided mapping table in starter kit).

GPS coordinates are linearly transformed to fall within the Casablanca bounding box.

This gives students authentic trip-length distributions, demand curves, and temporal patterns — without requiring a non-existent local dataset.

The remapping itself is a Week 1 data engineering task: students apply the transformation in PySpark and validate the output visually on an OSM map.

2.2 NYC TLC Trip Records — Spark Scale Dataset

Attribute	Detail
Full name	NYC Taxi & Limousine Commission Trip Record Data
Source	NYC Open Data / TLC (Public Domain) — also on Kaggle and AWS Open Data
URL	https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
Volume	~10M rows per month · Parquet format · 1 month ≈ 500 MB
Key fields	pickup/dropoff datetime · pickup/dropoff location IDs · trip distance · fare amount · passenger count
Usage in TaaSim	Batch processing layer only — Spark ETL, KPI computation, and ML training. Students use 3 months (~30M rows) to experience genuine large-scale Spark.
Why this dataset?	Freely available, well-documented, large enough to force real Spark optimization (partitioning, broadcast joins). Not used for streaming — Porto handles that.

2.3 Real-Time Simulation Layer

Since real-time Casablanca GPS feeds do not exist, the streaming layer is fed by a provided Python simulator that replays Porto trip data at accelerated speed through Kafka:

- **Replays Porto GPS polylines at 10× speed (one trip = seconds not minutes). Adds realistic noise: ±20m GPS drift, 5% chance of 60-second blackout per vehicle per trip.:** vehicle_gps_producer.py
- **Generates reservation events following Porto's demand curve (mapped to Casablanca time zones). Peak: 7–9am and 5–7pm. Friday and Sunday patterns included.:** trip_request_producer.py
- **Injects configurable anomalies: demand spike (stadium exit), GPS blackout burst, rain event (×1.4 demand multiplier). Used during live demo.:** event_injector.py

> **Key Engineering Constraint**
> The GPS producer deliberately emits out-of-order events with up to 3-minute delay and periodic blackouts.
> Students MUST implement event-time processing with watermarks in Flink.
> A processing-time approach will produce measurably incorrect demand aggregations — this is verified during evaluation.

3. Technology Stack

TaaSim is built on a carefully chosen set of industry-standard Big Data technologies. Each component plays a clearly defined role in the platform, and the boundaries between components are explicit. Students are expected to understand every layer and be able to defend architectural choices in their technical report and final demo.

3.1 Stack Overview

Layer	Technology	Role in TaaSim
Messaging	Apache Kafka (KRaft mode, 1 broker)	Central event bus: receives GPS pings and trip reservations; decouples producers from processing jobs; persists raw events for 7-day replay
Object Store	MinIO (S3-compatible, single node)	Distributed data lake: raw zone (CSV/JSON), curated zone (Parquet), ML zone (model artifacts). All Spark jobs read from and write to MinIO.
Batch + ML	Apache Spark (PySpark, local[*])	ETL on Porto and NYC historical data; feature engineering; GBT demand forecasting model training and evaluation (MLlib)
Streaming	Apache Flink (1 Job Manager + 1 Task Manager)	Real-time GPS normalization, demand aggregation (tumbling windows), and trip matching (stateful); RocksDB state backend; checkpointing to MinIO
Database	Apache Cassandra (single node)	Low-latency serving layer: vehicle positions, trip records, demand zone aggregates. Schema designed around API query patterns.
Dashboard	Grafana (Cassandra plugin)	Live operational dashboard: vehicle map, demand heatmap, KPI panels, ML forecast overlay
API	FastAPI (Python)	REST interface: trip reservation, vehicle lookup, demand forecast endpoint. JWT authentication with rider and admin roles.

Architecture Choice: Kappa

TaaSim is built on a Kappa Architecture: a single unified stream-processing pipeline where Kafka acts as the system of record.

Historical data is replayed through the same Kafka topics as live events — one processing engine (Flink) handles both cases.

Spark operates exclusively on the offline side: batch ETL and ML training, reading from and writing to MinIO.

> This separation of concerns — Flink for real-time, Spark for analytical depth — is a deliberate architectural choice students must understand and articulate.

3.2 Architecture Diagram

```
DATA SOURCES
┌──────────────────────────────────────────────────┐
│  Porto GPS Simulator   ·  Trip Request Simulator │
│  Porto Batch CSV       ·  NYC TLC Parquet (S3)   │
└──────────────────────────────────────────────────┘
                  │
        ┌─────────▼──────────┐
        │   Apache Kafka     │   ← 3 topics
        │  (KRaft, 1 broker) │     raw.gps
        └────────────────────┘     raw.trips
                 │      │           processed.demand
      ┌──────────▼──┐  ┌▼──────────────────────┐
      │  Flink      │  │  Kafka → MinIO S3 Sink │
      │  3 Jobs     │  │  (raw zone archival)   │
      └─────────────┘  └────────────────────────┘
        │  │                        │
   ETAs │  │ Demand            ┌────▼──────────┐
        │  │ zones             │    MinIO      │
        │  └──────────────►    │  raw/         │
        │                      │  curated/     │
        │              Spark   │  ml/          │
        │           ◄──────────└───────────────┘
        │
        │  Spark: ETL + MLlib demand forecast
        │                          │
      ┌─▼──────────────────────────▼──────┐
      │        Apache Cassandra           │
      │  trips · positions · demand_forecast│
      └───────────────────────────────────┘
                  │
            ┌─────▼──────────┐
            │  FastAPI  +  Grafana  │
            │  Live map · Heatmap   │
            │  KPI dashboard · Demo │
            └────────────────────────┘
```

3.3 Flink Processing Jobs

TaaSim's real-time pipeline is built around three focused Flink jobs, each responsible for a clearly delimited processing concern:

Job	Input Topic	Processing Logic	Output
Job 1 GPS Norm.	raw.gps	Validate coordinates · Remove duplicates · Assign event-time watermark (3-min allowed lateness) · Map-match to Casablanca zone grid	Cassandra: vehicle_positions · K afka: processed.gps

Job	Input Topic	Processing Logic	Output
Job 2 Demand Agg.	processed.gps + raw.trips	30-second tumbling window per zone · Count active vehicles + pending requests · Compute supply/demand ratio	Cassandra: demand_zones · Kafka: processed.demand
Job 3 Trip Match	raw.trips + processed.gps	On each trip request: find nearest available vehicle in same zone · Assign match · Compute simple ETA (distance ÷ avg speed) · Emit match event	Cassandra: trips · Kafka: processed.matches

What students learn from Job 1 alone

Event-time vs processing-time: GPS events arrive late — watermarks make this concrete, not theoretical.

Stateful deduplication: vehicles ping every 4 seconds; GPS noise can generate near-duplicate events.

Map-matching: raw lat/lon must be snapped to a zone grid (Casablanca arrondissements) — a real geospatial join.

This single job covers: Kafka consumer groups, Flink state backends, watermark strategy, and geospatial data.

4. Data Model — Cassandra Schema

Cassandra tables are designed around query patterns, not normalization. Students must understand this fundamental NoSQL design principle and justify every partition key choice in their technical report. The following three tables are the minimum required.

4.1 Required Tables

Table	Partition Key	Clustering Key	Key Fields	Primary Query
vehicle_positions	(city, zone_id)	event_time DESC	taxi_id · lat · lon · speed · status	Nearest available vehicles in zone
trips	(city, date_bucket)	created_at DESC	trip_id · rider_id · taxi_id · origin_zone · dest_zone · status · fare	Trip history by day
demand_zones	(city, zone_id)	window_start DESC	active_vehicles · pending_requests · ratio · forecast_demand	Live demand heatmap per zone

Partition Key Design — What students must justify

Why (city, zone_id) and not just taxi_id for vehicle_positions? Answer: the API queries 'all vehicles in zone X', not 'all trips by taxi Y'. Partition by query pattern.

Why date_bucket in trips? Answer: unbounded partition growth. Without bucketing by day/week, one partition accumulates millions of rows and causes hotspots.

Students who copy this schema without understanding it will not survive Q&A in Week 8.

4.2 MinIO Bucket Structure

Bucket / Prefix	Contents	Written by	Read by
raw/porto-trips/	Raw Porto CSV files as downloaded	Manual (Week 1)	Spark ETL
raw/nyc-tlc/	NYC TLC Parquet files (3 months)	Manual (Week 1)	Spark ETL
raw/kafka-archive/	Kafka S3 Sink mirror of all topics	Kafka Connect	Spark ETL
curated/trips/	Cleaned + geo-enriched trips (Parquet)	Spark ETL	Spark ML

Bucket / Prefix	Contents	Written by	Read by
ml/features/	Feature matrix for ML training	Spark ML Prep	Spark MLlib
ml/models/demand_v1/	Trained GBT model artifact (PipelineModel)	Spark ML Train	FastAPI

5. Spark ML Pipeline — Demand Forecasting

The ML component transforms TaaSim from a reactive dispatcher into a proactive platform. It is implemented in PySpark (MLlib) and trained on the cleaned Porto + NYC historical data. Scope is deliberately constrained: one model, one evaluation, one serving endpoint.

5.1 Problem Definition

Element	Definition
Prediction target	Number of trip requests per city zone over the next 30-minute time slot
Prediction horizon	30 minutes ahead
Granularity	Per Casablanca arrondissement (16 zones) × per 30-min slot
Algorithm	Gradient Boosted Trees regressor — Spark MLlib GBTRegressor
Training data	12 months Porto remapped trips (1.7M rows) + feature engineering
Baseline to beat	Naive model: predict same slot from 7 days ago (students must beat this)

5.2 Feature Set (Kept Simple)

Feature Group	Features	Engineering Step
Temporal	hour_of_day (0–23) · day_of_week (0–6) · is_weekend · is_friday	Extracted from TIMESTAMP in Spark
Spatial	zone_id (1–16) · zone_population_density · zone_type (residential / commercial / transit_hub)	Joined from provided zone reference table
Weather	is_raining (binary) · temperature_bucket (cold/mild/hot)	Joined from Open-Meteo historical API
Lag	demand_lag_1d · demand_lag_7d · rolling_7d_mean	Window functions over trip history in Spark

5.3 Training & Serving Steps

- **Spark job reads curated/trips/, computes all features, writes feature matrix to ml/features/:** Step 1 — Feature engineering
- **First 10 months = training, last 2 months = test. Temporal split — no data leakage:** Step 2 — Train/test split
- **GBTRegressor with maxDepth=5, maxIter=50. CrossValidator over 2 parameters only (not full grid search — time constraint):** Step 3 — Model training

- **RMSE and MAE on test set. Must be compared against naive 7-day-lag baseline. Feature importance chart required:** Step 4 — Evaluation
- **model.save('s3a://ml/models/demand_v1/') to MinIO:** Step 5 — Save artifact
- **FastAPI loads model at startup: POST /api/demand/forecast → {zone_id, datetime} → {predicted_demand}:** Step 6 — Serve via API

> **Evaluation Requirement**
>
> The ML model is only considered successful if it achieves lower RMSE than the naive 7-day-lag baseline.
> Students must include a table comparing model vs baseline per zone in their technical report.
> Feature importance chart must explain the top 3 predictors — what drives demand in Casablanca?

6. Requirements — Focused & Measurable

Requirements have been deliberately reduced to a set that is achievable by a 2–3 person team while still covering the essential non-functional dimensions of a production data platform. Each requirement has a concrete measurement method — 'it feels fast' is not acceptable.

6.1 Performance & Latency

Requirement	Target	How to Measure
Trip match latency (request → match event in Cassandra)	< 5 seconds P95	Log request timestamp in Kafka; compare to Cassandra write timestamp in Flink Job 3
Vehicle position freshness (GPS ping → Cassandra write)	< 15 seconds	Compare Kafka producer timestamp to Cassandra write timestamp in Job 1
Demand zone update frequency	Every 30 seconds	Verify Cassandra demand_zones rows using WRITETIME() function
ML forecast API response time	< 500ms at 20 req/s	locust load test script provided in starter kit
Spark ETL on full Porto dataset (1.7M rows)	< 5 minutes	Spark UI job duration

6.2 Reliability

Requirement	What to implement	Why it matters
Flink checkpointing	Enable checkpoint every 60 sec to MinIO; verify job recovers from manual task manager restart	Guarantees at-least-once processing; no lost GPS events on failure
Kafka consumer group offset	Use named consumer groups for all Flink jobs; verify offsets committed	Flink restart resumes from last checkpoint, not from Kafka topic start
Idempotent Cassandra writes	Use IF NOT EXISTS or upserts for trip matching writes	Prevents duplicate trip records from Flink at-least-once redelivery
Kafka topic retention	Set retention to 7 days on raw topics	Allows historical replay for ML feature recomputation without re-downloading data

6.3 Security (Minimal Viable)

Requirement	Implementation
API Authentication	JWT tokens on all FastAPI endpoints. Two roles only: rider (read + reserve) and admin (full access). Library: python-jose
GPS Anonymization	In Flink Job 1: snap raw lat/lon to zone centroid before writing to Cassandra. Raw coordinates never persisted.
Kafka Topic ACLs	Producers may only write to raw.* topics. Flink jobs read raw.* and write processed.*. Admin-only access to processed.demand.
HTTPS on API	Self-signed certificate acceptable for demo (not production). TLS termination at FastAPI with uvicorn --ssl-keyfile.

7. Weekly Lab Plan

Each week delivers one working layer of the platform. By Week 5 the core end-to-end pipeline is operational. Weeks 6 and 7 add intelligence and security hardening. Week 8 is the investor pitch and live demo.

Wk	Theme	Engineering Tasks	Deliverables
W1	**Setup & Data Exploration**	● Provision Docker Compose stack: Kafka (KRaft), MinIO, Cassandra, Flink, Spark, Grafana. ● Download Porto CSV (Kaggle) + NYC TLC Parquet (3 months) → upload to MinIO raw/. ● Explore Porto dataset in Jupyter: schema, trip duration distribution, call type breakdown, temporal demand curve. ● Apply zone remapping (Porto → Casablanca arrondissements) in PySpark. Visualize on OSM map. ● Start GPS + trip request Kafka producers. Verify events in Kafka console consumer.	✓ Docker stack running (screenshot). ✓ Jupyter data profiling notebook. ✓ Zone-remapped trips visualized on Casablanca map. ✓ Team name + 1-slide startup concept.
W2	**Storage Design**	● Finalize MinIO bucket structure: raw/ curated/ ml/ kafka-archive/. ● Configure Kafka Connect S3 Sink to mirror raw.gps and raw.trips to MinIO kafka-archive/. ● Design Cassandra keyspace: create vehicle_positions, trips, demand_zones tables. Justify partition keys. ● Write a 1-page Architecture Decision Record (ADR) documenting the storage and architecture choices made.	✓ MinIO buckets receiving data from Kafka. ✓ Cassandra schema deployed + documented. ✓ ADR submitted (architecture and storage rationale).
W3 ⚡	**Stream Processing I — GPS**	● Implement Flink Job 1 (GPS Normalizer): Kafka source, coordinate validation, zone mapping, watermark assignment (3-min allowed lateness), sink to Cassandra vehicle_positions. ● Test with injected late events: verify watermark handles 3-min late GPS correctly. ● Configure Flink checkpointing to MinIO every 60 seconds.	✓ Flink Job 1 running with checkpointing. ✓ Grafana shows live vehicle positions. ✓ Watermark test: late event handled correctly (documented).

Wk	Theme	Engineering Tasks	Deliverables
		• Connect Grafana to Cassandra: create live vehicle positions panel.	
W4	**Stream Processing II — Matching**	• Implement Flink Job 2 (Demand Aggregator): 30-second tumbling window per zone, write to demand_zones table. • Implement Flink Job 3 (Trip Matcher): on trip request event, match to nearest vehicle in zone, compute simple ETA, write match to trips table. • Implement 5-second fallback: if no vehicle in requested zone, expand to adjacent zones. • Verify end-to-end: reserve trip → match within 5 seconds → ETA returned by API stub.	✓ End-to-end trip flow: request → match → ETA < 5s. ✓ Grafana demand heatmap updating every 30s. ✓ Flink Job 3 state backend configured (RocksDB).
W5	**Batch ETL + Spark Analytics**	• Spark ETL job: read Porto CSV from MinIO raw/, apply zone remapping, deduplicate, compute H3 zone IDs, write Parquet to curated/. • Spark ETL on NYC TLC: read 3-month Parquet, compute per-zone demand aggregates, write to curated/. • Spark SQL analytics: compute weekly KPIs — trips per zone, avg trip duration, peak demand hours, coverage gap (zones with demand but < 2 vehicles). • Load KPI aggregates to Cassandra demand_zones for Grafana KPI panel.	✓ Spark ETL processes Porto 1.7M rows in < 5 min. ✓ Spark ETL processes NYC 10M rows/month. ✓ Grafana KPI panel: corridor demand, peak hours.
W6	**ML — Demand Forecasting**	• Feature engineering Spark job: extract temporal, spatial, weather, and lag features from curated/trips/. • Train GBTRegressor on 10-month training set. Evaluate on 2-month test set. Compare vs 7-day-lag baseline. • Save model artifact to MinIO ml/models/demand_v1/. • Implement FastAPI endpoint: POST /api/demand/forecast returns predicted demand per zone. • Display ML forecast overlay on Grafana demand heatmap.	✓ Model RMSE beats naive baseline. ✓ Feature importance chart (top 3 predictors explained). ✓ API /demand/forecast responds < 500ms.

Wk	Theme	Engineering Tasks	Deliverables
W7	**Security + Integration**	• Add JWT authentication to FastAPI: rider and admin roles. Test with curl. • Implement GPS anonymization in Flink Job 1: snap coordinates to zone centroid. • Full integration test: run all 3 Flink jobs + Spark ETL + API simultaneously for 30 minutes. • Measure all SLA targets from Section 6.1. Record results for technical report. • Kill Flink task manager manually → verify job recovers from checkpoint.	✓ JWT auth working on all endpoints. ✓ SLA measurement table completed. ✓ Checkpoint recovery demonstrated (screen recording).
W8	**Demo + Investor Pitch**	• Polish Grafana dashboard: vehicle map, demand heatmap, trip funnel, ML forecast vs actual. • Rehearse live demo script: inject morning rush → demand spike from event_injector → show trip matches + heatmap response. • Prepare 10-slide pitch deck: Problem · Solution · Architecture · Live Demo · Metrics · Business model · Team. • Finalize technical report (12–15 pages).	✓ Live 20-minute demo + 10-min Q&A. ✓ Technical report submitted. ✓ Pitch deck submitted.

8. Evaluation Rubric

Evaluation is designed so that a team of 2 working consistently can achieve a good result. Distinction requires both technical depth and entrepreneurial clarity. The live demo is non-negotiable — a pipeline that does not run on Demo Day cannot pass the pipeline pillar.

Pillar	Weight	Passing Level	Distinction Level
Pipeline Completeness	30%	All 3 Flink jobs running; Spark ETL completes; Cassandra tables populated; trips can be reserved via API.	All SLA targets from §6.1 met and measured. Checkpoint recovery demonstrated. Anomaly injection handled gracefully.
Engineering Quality	25%	Flink checkpointing configured; GPS anonymization implemented; JWT auth on API; Kappa vs Lambda justified in report.	Partition key choices justified with query benchmarks. Watermark strategy explained with late-event evidence. ADRs for all major decisions.
Technical Report (12–15p)	25%	Architecture described. Dataset remapping documented. ML evaluation includes baseline comparison.	Honest post-mortem on failures. Feature importance explained in business terms. NFR measurement table included.
Startup Pitch + Demo	20%	Pipeline runs live. Team explains every component when asked. Business model is coherent.	Demand spike injected live and handled visibly. Pitch is compelling to a non-technical observer. Team defends tradeoffs under pressure.

8.1 Minimum Viable Pipeline — Demo Day Checklist

✓ The 5 things that must work on Demo Day

1. GPS events flowing: Kafka → Flink Job 1 → Cassandra → Grafana vehicle position map (updating live).
2. Trip reservation: POST /api/trips → Flink Job 3 match → trip record in Cassandra with ETA < 5 seconds.
3. Demand heatmap: Flink Job 2 → Cassandra demand_zones → Grafana heatmap updating every 30 seconds.
4. ML forecast: Spark-trained model → FastAPI /demand/forecast responding in < 500ms.
5. Anomaly visible: event_injector.py demand spike → heatmap shows surge zone within 60 seconds.

8.2 Extension Challenges (for groups finishing early)

Challenge	Bonus	Description
Schema evolution	+3%	Add accessibility_required field to trip request Avro schema. Demonstrate backward-compatible consumer using Confluent Schema Registry.
Driver earnings tracker	+3%	Per-driver weekly earnings and utilization rate, updated in real time from Flink Job 3 output. New Cassandra table + Grafana panel.
Dynamic pricing	+4%	Flink job adjusts fare multiplier based on supply/demand ratio per zone. Price capped at 2× base. Displayed in API response and dashboard.
Kappa vs Lambda report	+4%	Implement the demand aggregation in Spark Streaming AND Flink. Benchmark: latency, throughput, LOC, operational complexity. 2-page ADR.
Real Porto coordinates	+2%	Deploy a second dashboard view showing actual Porto geography alongside the Casablanca remapping. Discuss what the remapping preserves and distorts.

9. Technical Guidance & Resources

This section provides the key technical indications students need to build each component of the platform. The guidance is intentionally directional — it points to the right approach and the key decision points — but leaves the implementation to the team. This is what engineering judgment means in practice.

9.1 Infrastructure Setup

Docker Compose Stack

All services must run via Docker Compose on a single workstation (8 GB RAM minimum).

Services to include: Kafka (KRaft mode — no Zookeeper), MinIO, Apache Cassandra, Apache Flink (1 Job Manager + 1 Task Manager), Apache Spark (master + 1 worker), Grafana.

Key configuration points: Flink and Spark both need access to MinIO via the S3A connector (hadoop-aws JAR + s3a:// paths). Cassandra needs its CQL port exposed. Grafana needs the Cassandra datasource plugin installed.

Health check: verify each service with a simple client command before proceeding to Week 2 (kafka-topics.sh --list, mc ls, cqlsh, flink list, spark-shell, Grafana UI on port 3000).

9.2 Data Producers (Streaming Simulation)

Script to build	Purpose	Key implementation hints
vehicle_gps_producer.py	Replays Porto GPS polylines through Kafka topic raw.gps at accelerated speed	Read Porto POLYLINE field (JSON array of [lon, lat] pairs). Iterate coordinates at configurable speed (e.g. 10× real time). Apply coordinate transform: map Porto bounding box linearly to Casablanca bounding box (provided in zone_mapping.csv). Add noise: Gaussian jitter ($\sigma \approx 0.0002$ degrees ≈ 20m). With 5% probability per vehicle per event, delay the Kafka send by 60–180 seconds to simulate blackouts. Kafka message key = taxi_id. Payload fields: taxi_id, timestamp (event time), lat, lon, speed, status.
trip_request_producer.py	Generates citizen reservation events following Porto's demand curve on Kafka topic raw.trips	Compute a demand multiplier per hour from the Porto dataset (aggregate trips by hour → normalize). Apply this curve to control event emission rate. Each event: trip_id (UUID), rider_id, origin_zone, destination_zone, requested_at (event time), call_type (A/B/C from Porto CALL_TYPE).

Script to build	Purpose	Key implementation hints
		Peak hours (7–9h, 17–19h) should produce ≈ 3–5× the off-peak rate. Friday 12–14h should have reduced rate.
event_injector.py	Injects configurable demand anomalies for live demo	Demand spike: for a chosen zone, multiply emission rate by a configurable factor (e.g. 3.0) for 5 minutes. Simulates a stadium exit or train cancellation. GPS blackout: suppress all GPS events from a set of vehicles for a configurable duration. Rain event: increase trip request rate globally by 1.4× for a configurable period. Design as a standalone script that publishes synthetic events directly to the same Kafka topics.

9.3 Flink Jobs

Job	Key implementation hints
Job 1 — GPS Normalizer	Kafka source: use FlinkKafkaConsumer with Avro or JSON deserializer on raw.gps topic. Assign BoundedOutOfOrdernessWatermarks with 3-minute max lateness. Validation: filter out coordinates outside Casablanca bounding box (lon: 33.4–33.7°N, lat: 7.4–7.8°W). Discard speed > 150 km/h. Zone mapping: broadcast the zone_mapping reference table as a broadcast state. For each GPS event, determine which arrondissement it falls in using bounding box lookup. Anonymization: replace raw lat/lon with zone centroid coordinates before writing to Cassandra. Raw coordinates must not be persisted. Sink: CassandraAppendTableSink to vehicle_positions table. Also forward to processed.gps Kafka topic for Job 2.
Job 2 — Demand Aggregator	Dual Kafka source: consume both processed.gps (vehicle positions) and raw.trips (pending requests). Use event-time processing on both streams. Tumbling window of 30 seconds per zone_id. In each window: count distinct active vehicles + count pending trip requests → compute ratio = requests / max(vehicles, 1). Key the stream by zone_id before windowing to parallelise per zone. Sink: upsert to Cassandra demand_zones table (use zone_id + window_start as composite key). Also publish to processed.demand Kafka topic.
Job 3 — Trip Matcher	Consume raw.trips. For each new trip request event, perform a point lookup into Flink's keyed state (keyed by zone_id) to find available vehicles. Matching logic: select the vehicle in the same zone with status=available and oldest last_seen timestamp. If no vehicle in zone, expand search to adjacent zones (adjacency list provided in zone_mapping.csv). SLA: if no match found within 5 seconds of event time, emit an unmatched event to a separate Kafka topic for monitoring.

Job	Key implementation hints
	On match: update vehicle status to assigned in Flink state; emit match event with trip_id, taxi_id, estimated_arrival_seconds (= distance_km / avg_speed_kmh × 3600). Write trip record to Cassandra trips table.

9.4 Spark Jobs

Job	Key implementation hints
ETL — Porto + NYC	Read Porto CSV from MinIO (s3a://raw/porto-trips/). Parse POLYLINE JSON column using from_json + explode to produce one row per GPS point. Apply zone remapping: join on a zone_mapping broadcast DataFrame to add arrondissement_id and zone_type to each trip record. Deduplication: drop rows where MISSING_DATA = True (Porto field). Also deduplicate on TRIP_ID. NYC TLC: read Parquet from s3a://raw/nyc-tlc/. Compute per-zone-per-hour demand counts using groupBy + agg. Write to s3a://curated/demand-by-zone/. Write cleaned Porto trips to s3a://curated/porto-trips/ in Parquet with snappy compression. Partition by year_month for efficient ML reads.
ML — Demand Forecasting	Feature engineering: for each (zone_id, time_slot_30min) compute: hour_of_day, day_of_week, is_weekend, is_friday, demand_lag_1d, demand_lag_7d, rolling_7d_mean (use Spark Window functions). Join with Open-Meteo weather data (fetch historical Porto weather matching trip timestamps): add is_raining (bool), temperature_bucket (cold < 15°C / mild / hot > 28°C). Join with zone reference table: add zone_population_density, zone_type (one-hot encoded). Train/test split: first 10 months = train, last 2 months = test. Use filter on year_month — never shuffle across time. Pipeline: VectorAssembler → StandardScaler → GBTRegressor. Wrap in CrossValidator with 3 folds and 2 maxDepth values (5, 7) only. Evaluation: compute RMSE and MAE on test set. Compare to naive baseline: predict demand_lag_7d. Model passes only if it beats the baseline. Save: model.write().overwrite().save('s3a://ml/models/demand_v1/'). Log feature importances to a text file in MinIO for the report.

9.5 FastAPI Service

Key endpoints and implementation hints

POST /api/v1/trips — Body: {origin_zone, destination_zone, rider_id}. Publishes event to raw.trips Kafka topic. Returns {trip_id, status: pending}.

GET /api/v1/trips/{trip_id} — Reads from Cassandra trips table. Returns match status and ETA once Job 3 has processed the event.

GET /api/v1/vehicles/zone/{zone_id} — Queries Cassandra vehicle_positions for all vehicles in zone with event_time > now - 30s.

POST /api/v1/demand/forecast — Body: {zone_id, datetime}. Loads PipelineModel from MinIO at startup (cache in memory). Returns {predicted_demand, zone_id}.

JWT: use python-jose library. Generate tokens with /auth/token endpoint (username + role in payload). Verify via FastAPI Depends() on each route. Rider tokens cannot access /vehicles or /demand/forecast.

9.6 Grafana Dashboard

Panel setup guidance

Datasource: install Grafana Cassandra plugin (HadesArchitect-Cassandra-datasource). Configure keyspace = taasim.

Panel 1 — Vehicle map: use Geomap panel type. Query vehicle_positions WHERE event_time > now - 30s. Map taxi_id to point, colour by status (available=green, assigned=orange).

Panel 2 — Demand heatmap: use Geomap with heatmap layer. Query demand_zones WHERE window_start > now - 2min. Colour intensity = pending_requests / active_vehicles ratio.

Panel 3 — KPI table: query Cassandra for trips in last 24h. Show: total trips, avg ETA, % matched within 5s, top 3 demand zones.

Panel 4 — ML forecast overlay: bar chart per zone comparing demand_zones.pending_requests (actual) vs demand_zones.forecast_demand (ML prediction from Flink Job 2 enrichment).

Refresh interval: set dashboard to auto-refresh every 10 seconds during demo.

9.7 Dataset Access

- **https://www.kaggle.com/c/pkdd-15-predict-taxi-service-trajectory-i (free Kaggle account required):** Porto Taxi Trajectories (ECML 2015)
- **https://archive.ics.uci.edu/ml/datasets/Taxi+Service+Trajectory:** Porto dataset — UCI mirror
- **https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page (direct Parquet download, no login):** NYC TLC Trip Records
- **https://download.geofabrik.de/africa/morocco.html:** OpenStreetMap Casablanca extract
- **https://open-meteo.com/en/docs/historical-weather-api:** Open-Meteo historical weather (free API)
- **https://data.gov.ma:** Morocco open data portal

9.8 Recommended Reading

- **Designing Data-Intensive Applications — Ch. 10–12 (stream processing, Lambda vs Kappa). O'Reilly, 2017. Essential.:** Kleppmann, M.
- **Stateful Stream Processing + Watermarks: https://flink.apache.org/docs/stable/:** Flink documentation
- **Official guide: https://cassandra.apache.org/doc/latest/cassandra/data_modeling/:** Cassandra Data Modeling

- **Moreira-Matias et al., 'Predicting Taxi-Passenger Demand Using Streaming Data' — IEEE Trans. on Intelligent Transportation Systems, 2013. Directly uses the Porto dataset.:** ECML 2015 paper
- **The Lean Startup — startup methodology, cold start problem, MVP thinking. Non-technical but required for entrepreneurial framing.:** Ries, E.

TaaSim · Advanced Big Data Capstone · ENSA Al Hoceima · 2025–2026

"The best time to build the data infrastructure for Moroccan mobility was 10 years ago. The second best time is now."